Machine Translated by Google

[Chap 13] Recurrent Neural Networks and Time Series Forecasting

Machine Translated by Google

Time-series data and RNN

- Time series data: Data that depends on the chronological order
  - Stock prices, interest rates, product demand, temperature, blood pressure, voice signals … – Text in language also corresponds to time series data • Characters appear sequentially from the beginning, influencing the interpretation of the next character

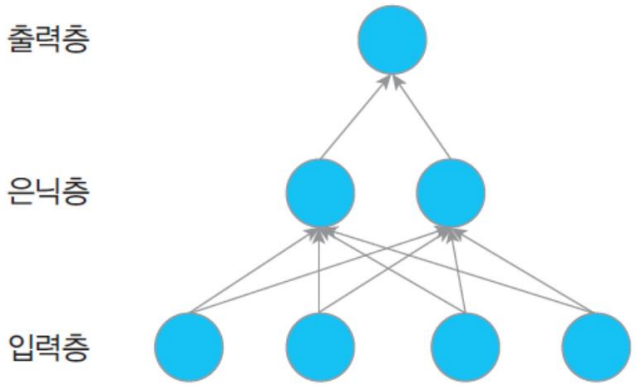
- Chronological observation inherently relies on 'memory' of past data – to predict a stock price, one must 'remember' the process by which a stock that was 30,000 won two weeks ago has risen to 50,000 won today in order to predict the next price – in a simple neural network structure, there is no mechanism capable of maintaining memory over time doesn't exist

- RNN (Recurrent Neural Network)
  - Maintain memories over time and interpret time-series data based on them Neural network structure designed for prediction

Machine Translated by Google

Traditional neural networks

- Feed-forward Neural Network
  – Output values are calculated sequentially from the input through the hidden layer in a unidirectional manner – There is no device to remember what the previous data was when the next data comes in.

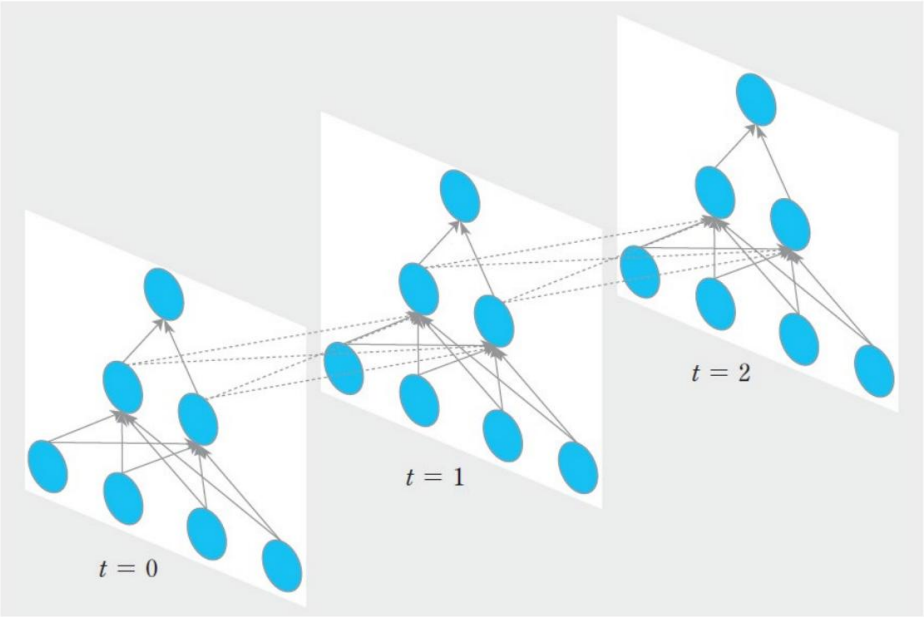


[그림 13–1] 단순한 인공신경망의 예

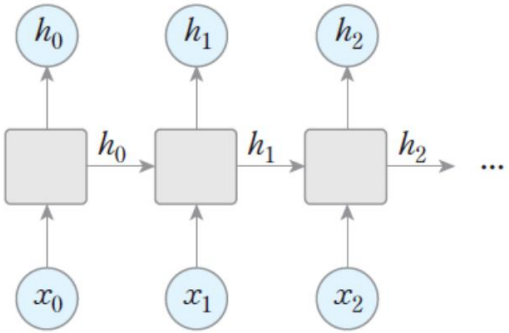
Machine Translated by Google

RNN

- When the next input arrives, not only the input data but also the output value of the previously held hidden layer is considered as input –
  the output value of the existing hidden layer is called the hidden state.



[그림 13–2] RNN의 개념



[그림 13–3] RNN의 도식화

- RNN Training: BPTT (Back-Propagation Through Time) – Extending backpropagation to work backward along the time axis

Machine Translated by Google

Limitations of Simple RNNs

- Long-term dependency problem due to vanishing gradient
  - Simple RNNs undergo backpropagation over a certain period of time or longer while traversing the time axis. In this case, the gradient becomes 0, so distant past data cannot be considered.

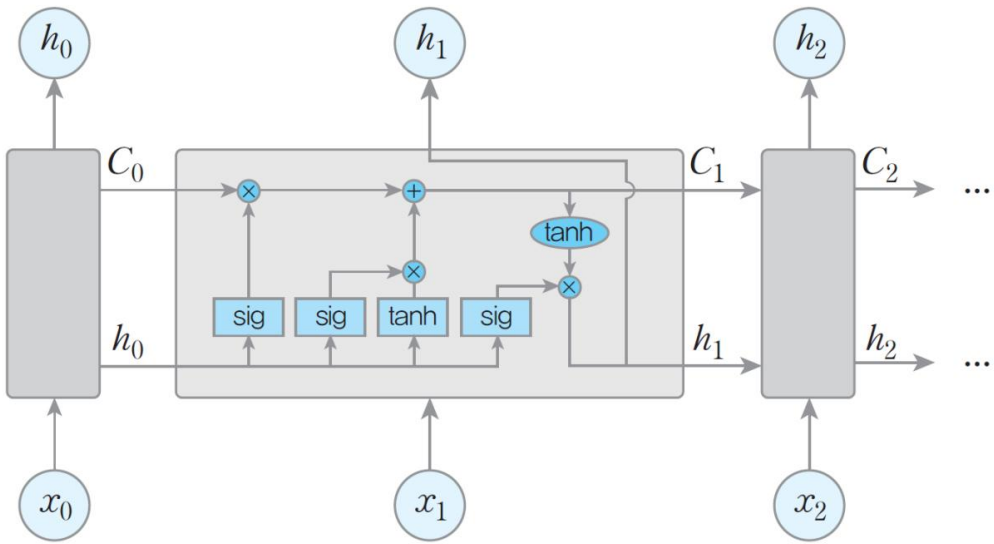
- Data from a long time ago often becomes important later on – It rained so I opened the ( ) ÿ You can easily deduce the 'umbrella' inside the parentheses from the preceding 'rain' – I spent my childhood in Busan … Even now, my ( ) dialect just won't go away ÿ You must remember the information 'Busan' from way back to infer the 'Busan' inside the parentheses

- Simple RNNs have limitations because past memories are sometimes important.
  - Improved RNN models designed to retain both short-term and long-term memory have been proposed
  - LSTM (Long-Short Term Memory), GRU (Gated Recurrent Unit) …

Machine Translated by Google

LSTM (Long Short Term Memory)

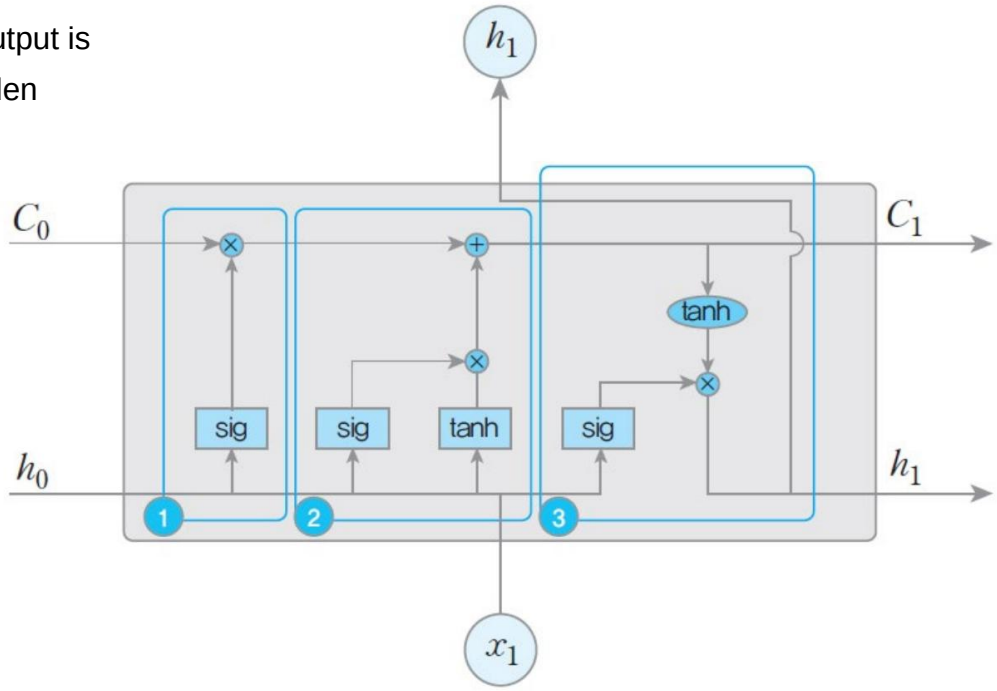
- Implementation of a mechanism capable of retaining long-term memory while also forgetting if necessary
  - There are two links that go over time.
    - The link at the bottom represents the previous hidden state, similar to an RNN. •
    The link added at the top represents the cell state, enabling the long-term transfer of memories.



Machine Translated by Google

Operation of LSTM (1)

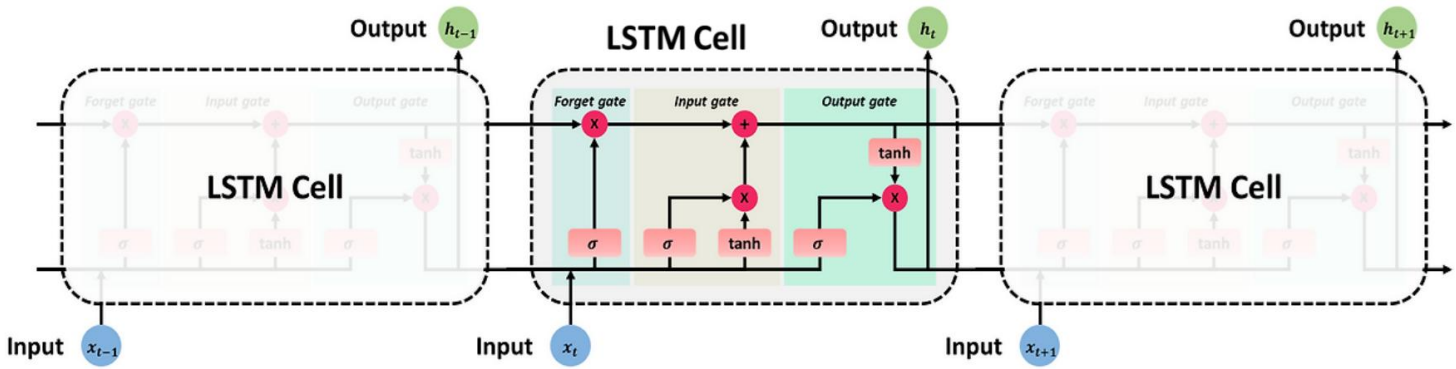
- 1: Forget Gate – Deletes unnecessary long-term memory by multiplying the cell state (0) by the sigmoid output (sig).
- 2 : Input gate
  - Add new information to cell state
- 3 : Calculate output value (Output gate)
  - The cell state (0) is output as is, and the output is generated by reflecting the previous hidden state h0 and the new data 1 into 0.



[그림 13-5] LSTM의 작동

Machine Translated by Google

Operation of LSTM (2)

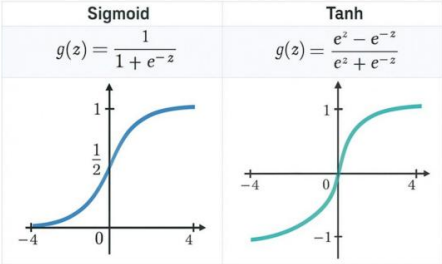


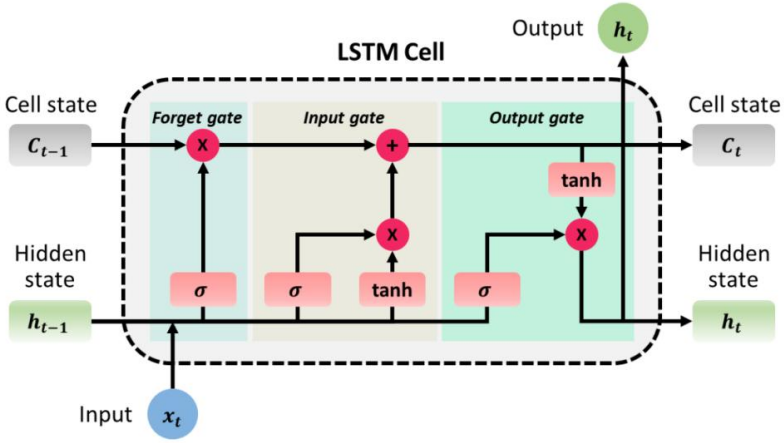
- The initial hidden state (h0) and cell state (0) are both initialized to 0 (zero) by default – Cell state:

  Responsible for long-term memory • At the Forget gate, some long-term information is erased, and at the Input gate, some information is newly accumulated –

  Hidden state (h): Responsible for short-term memory • Summarizes the short-term state in each cell and passes it as short-term information to the next cell

- 2 activation functions
  - Tanh: (-1 to 1), used in calculating the output of stored alc
  - Sigmoid(ÿ): (0~1), calculates the ratio of memory, storage, and output

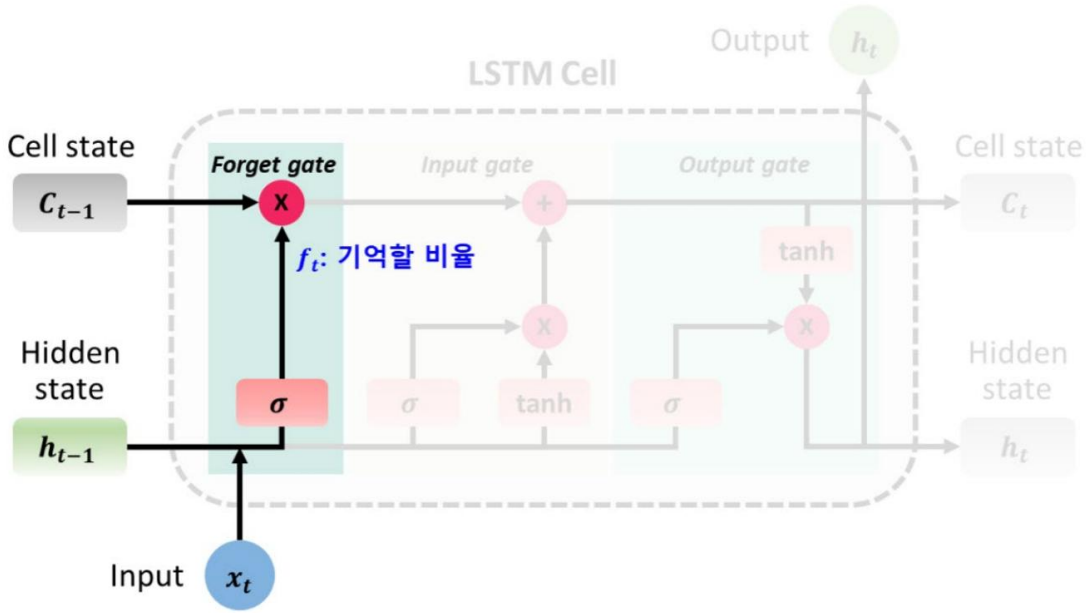




Machine Translated by Google

Operation of LSTM (3)

- Forget Gate: Determines what information from the previous cell state to forget and for how long.



– Calculate the output of sigmoid(ÿ) for inputs ÿ ÿ1 (where ÿ represents the proportion of ÿ1 to remember) – Calculate the inner product (x) of ÿ1 and ÿ

$$f_t = \sigma\left(W_{fx}x_t + W_{fh}h_{t-1} + b_f\right)$$

$$f_t \otimes C_{t-1}$$

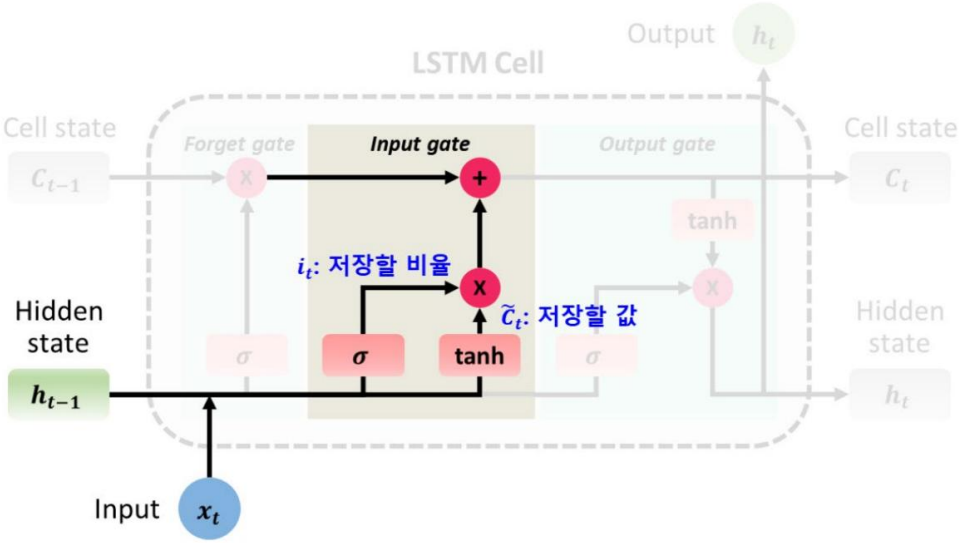
(example)

C_{t-1}					f_t					$C_{t-1} \otimes f_t$			
0.2	-1	1.35	3.3	$\otimes$	1	0.6	0	0.3	=	0.2	-0.6	0	1.1
					I remember everything	60% memory	Forgetting everything	30% memory					

Machine Translated by Google

Operation of LSTM (4)

- Input Gate: Adds new input information to the cell state and stores it.



– (calculate the new information value to be stored by calculating (x) each of input and . Calculate and with ÿ and tanh, respectively, for h ÿ1). $\tilde{C}_t$

– The value transmitted from the Forget Gate and the result calculated at the Input Gate become $i_t \otimes \tilde{C}_t$ the (+) operation the long-term memory of the next Cell. C_t

$$i_t = \sigma\left(W_{ix}x_t + W_{ih}h_{t-1} + b_i\right)$$

$$\tilde{C}_t = \tanh(W_{cx}x_t + W_{ch}h_{t-1} + b_c)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

(example)

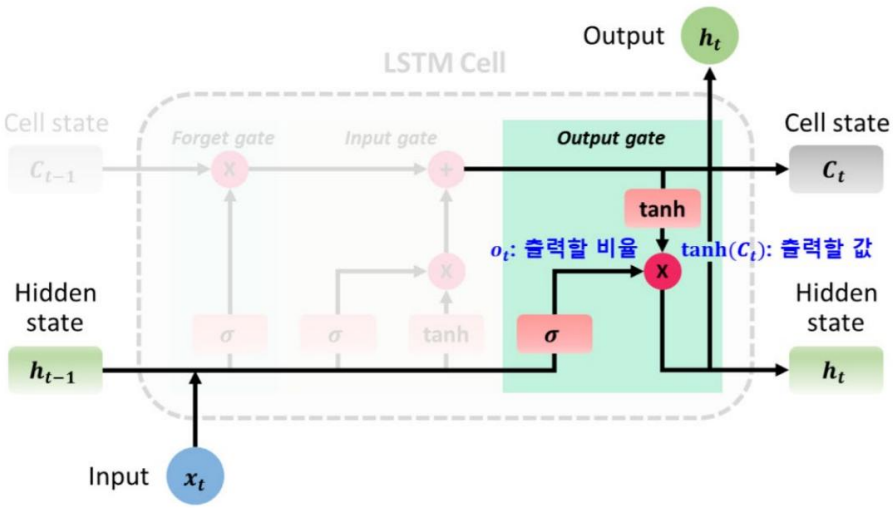
$\tilde{C}_t$					i_t					$\tilde{C}_t \otimes i_t$			
0.5	0.15	0.2	0.9	$\otimes$	0.8	0	0.1	0.5	=	0.4	0	0.02	0.45
					80% 저장	10% 저장 안함	10% 저장	50% 저장					

$C_{t-1} \otimes f_t$					$\tilde{C}_t \otimes i_t$					C_t			
0.2	-0.6	0	1.1	$\oplus$	0.4	0	0.02	0.45	=	0.6	-0.6	0.02	1.55

Machine Translated by Google

Operation of LSTM (5)

- Output Gate: Current Cell output value h ÿ1 calculate



– (input , Calculate (ratio to be output) as ÿ for h ÿ1)

– Apply tanh to the value calculated at the Input Gate to scale to a value between -1 and 1.

– Calculate h by performing the (x) operation with tanh()

$$o_t = \sigma\left(W_{ox}x_t + W_{oh}h_{t-1} + b_o\right)$$
$$h_t = o_t \otimes \tanh(C_t)$$

(example)

$\tanh(C_t)$					o_t					h_t			
0.54	-0.54	0.02	0.91	⊗	0	0.5	1	0.5	=	0	-0.27	0.02	0.46
					출력 안함	50% 출력	모두 출력	50% 출력					

Machine Translated by Google

Stock price prediction example using LSTM

- Install FinanceDataReader package (pip install finance_datareader)
  - Supports importing stock price data by stock ticker or code (6-digit KOSPI code)

명령 프롬프트

```
Microsoft Windows [Version 10.0.19044.1826]
(c) Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>pip install finance-datareader
```

- Example) Retrieve Coca-Cola stock price (ticker KO) from 2010 to 2019
  - In addition to the date, display Close, Open, High, Low, Volume, and Change in price compared to the previous day

```python
import FinanceDataReader as fdr
stock = fdr.DataReader('KO', '2010-01-01', '2019-12-31')
print(stock)
```

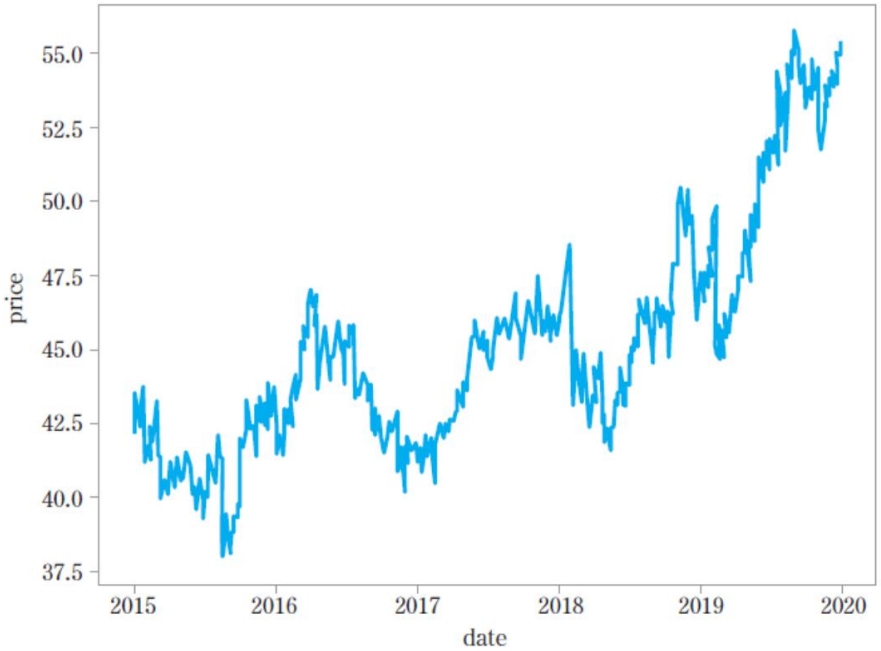
```
            Close   Open   High    Low      Volume  Change
Date
2015-01-02  42.14  42.26  42.40  41.80   9920000.0 -0.0019
2015-01-05  42.14  42.69  42.97  42.08  26290000.0  0.0000
2015-01-06  42.46  42.41  42.94  42.24  16900000.0  0.0076
...            ...    ...    ...    ...         ...     ...
2019-12-31  55.35  55.20  55.38  54.98   7980000.0  0.0014

[1258 rows x 6 columns]
```

Machine Translated by Google

Visualization of Close Price

```
import numpy as np
prices = np.asarray(stock['Close'], dtype='float32')
import matplotlib.pyplot as plt
plt.figure()
plt.plot(stock.index, prices)
plt.xlabel('date')
plt.ylabel('price')
plt.show()
```



Machine Translated by Google

Data normalization

- Adjust the input data to a value between 0 and 1.
  - Neural network inputs must be adjusted to the range between 0 and 1 for stable operation

$$y' = \frac{y - \min(y)}{\max(y) - \min(y)}$$

```
y = (prices - min(prices))/(max(prices)-min(prices))
print(y)
```

```
[0.23340821 0.23340821 0.25140595 ... 0.97637784 0.9718785  0.97637784]
```

Machine Translated by Google

Time series data preparation

- To predict the next day's stock price based on the closing price of the previous 60 days

- If there are 1,000 records, the number of data points will be (1,000 – 60).

첫번째 데이터	0	1	2	...	59	→	60
두번째 데이터	1	2	3	...	60	→	61
세번째 데이터	2	3	4	...	61	→	62
...							
940번째 데이터	939	940	941	...	998	→	999

[그림 13-7] 시계열 학습을 위한 데이터 구조

Machine Translated by Google

Time Series Data Preparation (Continued)

- Convert closing price data in the form of a one-dimensional array into a structure for time series learning

```
1  window_size = 60
2  data_length = len(y) - window_size
3  y_data = np.zeros(data_length, dtype='float32')
4  x_data = np.zeros((data_length, window_size, 1), dtype='float32')
5  for i in range(0, data_length):
6      y_data[i]=y[i+window_size]
7      x_data[i]=y[i:i+window_size].reshape(window_size,1)
```

– (Lines 1–2) window_size=60, data_length =len(y) – Set to window_size – (Line 3) Initialize the ground truth values to 0 as an array of length data_length – (Line 4) Initialize by defining the structure of the input data • Keras LSTM takes a structure of (number of periods × number of features) as input for each data record • Since the period is 60 days and there is only one feature, the closing price, each data record takes the form of (60 × 1), and the entire data is a 3D array structure of (data_length × 60 × 1).

– (Lines 5–7) Specify the correct answer and input values for each data point

• The correct answer value y_data[i] is set to the stock price y[I + window_size] 60 days later. • The input value x_data[i] contains the stock prices from i to i+59. – Since x_data[i] must be a 2D (window_size, 1), apply reshape(window_size, 1).

Machine Translated by Google

Time Series Data Preparation (Continued)

- Split into training and test data – Use 30%
  as test data – Since order is
  important for time series data, specify shuffle=False to prevent random extraction of test data

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x_data, y_data,
 test_size=0.3, shuffle=False)
```

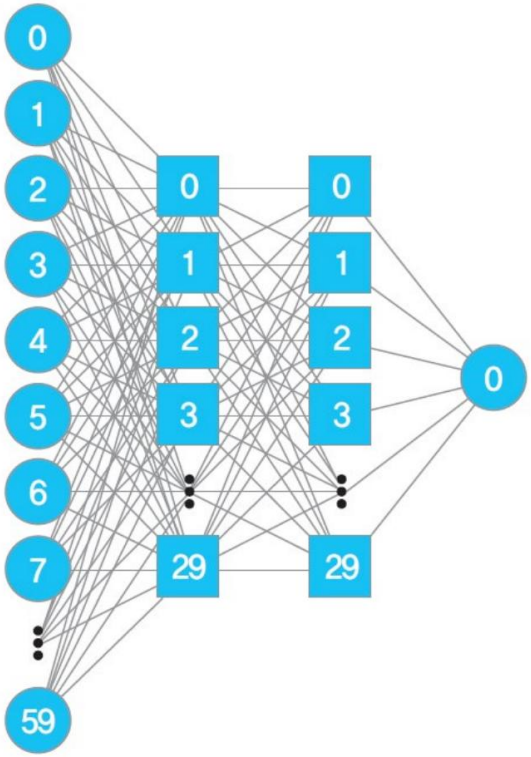
Machine Translated by Google

Neural network modeling

- Neural Network

  Architecture Design – 60

  inputs, 1 output – Configured with 2 hidden layers, each containing 30 LSTMs.



[그림 13-8] 주가 예측을 위한 인공신경망 구조

Machine Translated by Google

Keras modeling code

- Model the previous structure with Keras

```
1  model = keras.Sequential([
2      keras.layers.LSTM(30, return_sequences = True, input_shape=(window_size, 1)),
3      keras.layers.LSTM(30),
4      keras.layers.Dense(1, activation=None),
5  ])
```

– (Line 2) Set 30 LSTMs, and add the return_sequences=True option since the sequential output values of each hidden state over 60 days must be passed to the next layer. –

(Line 3) Set the second LSTM to 30. – (Line

4) The final output layer is single-value and must output continuous values, so set the activation function to None (activation=None).

- The optimization algorithm is Adam, and since the output values are continuous, the loss function is 'mse' (mean squared) Designated as error)

```
model.compile(optimizer='adam', loss='mean_squared_error')
model.fit(x_train, y_train, epochs=30)
```

Machine Translated by Google

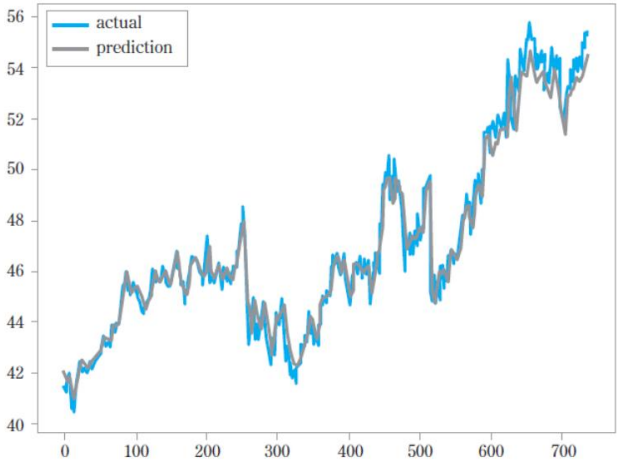
Prediction and Visualization

- Prediction execution and visualization (Full code: ex13-1.py)

```
pred = model.predict(x_test)

pred_price = (max(prices)-min(prices))*pred + min(prices)
target_price = (max(prices)-min(prices))*y_test + min(prices)

plt.figure()
plt.plot(target_price, label='actual')
plt.plot(pred_price, label='prediction')
plt.legend()
plt.show()
```



[그림 13-9] LSTM 기반 인공신경망의 주가 예측 결과

– Please note that the above prediction results forecast the next day's stock price based on the previous 60 days' prices, so they fundamentally tend to follow the

trend. – For actual predictions, a sophisticated model is required that considers various information such as corporate performance, market reaction, and economic conditions.

Machine Translated by Google

Full code

예제 ex13-1.py

```
import FinanceDataReader as fdr
import numpy as np
import matplotlib.pyplot as plt

stock = fdr.DataReader('KO', '2010-01-01', '2019-12-31')
prices = np.asarray(stock['Close'], dtype='float32')
y = (prices - min(prices))/(max(prices)-min(prices))

window_size = 60
data_length = len(y) - window_size
y_data = np.zeros(data_length, dtype='float32')
x_data = np.zeros((data_length, window_size, 1), dtype='float32')
for i in range(0, data_length):
    y_data[i]=y[i+window_size]
    x_data[i]=y[i:i+window_size].reshape(window_size,1)
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x_data, y_data,
 test_size=0.3, shuffle=False)

from tensorflow import keras
model = keras.Sequential([
    keras.layers.LSTM(30, return_sequences = True, input_shape=(window_size, 1)),
    keras.layers.LSTM(30, input_shape=(window_size, 1)),
    keras.layers.Dense(1, activation=None),
])
model.compile(optimizer='adam', loss='mean_squared_error')
model.fit(x_train, y_train, epochs=30)
pred = model.predict(x_test)

pred_price = (max(prices)-min(prices))*pred + min(prices)
target_price = (max(prices)-min(prices))*y_test + min(prices)

plt.figure()
plt.plot(target_price, label='actual')
plt.plot(pred_price, label='prediction')
plt.legend()
plt.show()
```